AIDS-2 Mini Project

AI-Based Document Question Answering using RAG

(Tauji)

Sr. No.	Student Name	Roll Number
1	Abdur R. Qureshi	242466
2	Haadiya Sayed	242467

Guide Name	Dr. Zainab Mirza

Abstract

Tauji is an AI-based Document Question Answering system that lets users ask natural-language questions about PDF and DOCX files. It uses Retrieval-Augmented Generation (RAG): document text is extracted, chunked, and indexed with TF-IDF; cosine similarity retrieves relevant sections; and the Qwen3 4B model, running locally via Ollama, generates context-based answers. Built with Python and Streamlit, it also shows source sections for verification. Tauji simplifies document search and provides a locally running AI assistant for document-based Q&A.

Introduction

Digital documents hold large amounts of information, making manual searches difficult. Traditional keyword searches often miss relevant content when a question's wording differs from the document.

Tauji lets users upload a PDF or DOCX and ask questions in natural language. It retrieves relevant text sections and uses a language model to answer.

The system follows a RAG approach: TF-IDF represents document chunks numerically, cosine similarity ranks their relevance, and Qwen3 4B generates the final answer from the retrieved context.

Problem Statement

Manually searching long documents is slow and inefficient, and keyword-based search may fail when users phrase questions differently from the document. Cloud-based AI services may also require external API access and send document content off the user's machine.

The objective is therefore to build a locally running document question-answering system that retrieves relevant information from uploaded documents and generates answers from the available context.

Objectives

- To support PDF and DOCX document formats.
- To extract and divide document text into smaller chunks.
- To implement TF-IDF-based text representation.
- To retrieve relevant chunks using cosine similarity.
- To generate answers using a locally hosted language model.
- To display relevant document sources for verification.

Working

Document Upload and Text Extraction

The user uploads a PDF or DOCX document through the Streamlit interface. PyPDFLoader is used to extract text from PDF files, while Docx2txtLoader extracts text from DOCX files.

Text Chunking

The extracted text is divided into smaller chunks using `RecursiveCharacterTextSplitter`. The current configuration uses a chunk size of approximately 2,000 characters with an overlap of 100 characters. Chunking makes the document easier to search and allows the system to retrieve smaller, relevant sections.

TF-IDF Vectorization

`TF-IDF` stands for **Term Frequency–Inverse Document Frequency**. It assigns numerical weights to terms based on their frequency within a text and their distribution across the collection of chunks.

`Term Frequency` (TF) measures how frequently a term occurs in a chunk. `Inverse Document Frequency` (IDF) reduces the importance of terms that occur across many chunks.

The resulting `TF-IDF` vectors provide a numerical representation of the document text.

Cosine Similarity

When a user submits a question, it is transformed using the same fitted `TF-IDF` vectorizer. `Cosine similarity` measures the similarity between the question vector and the vectors of document chunks.

The system ranks chunks by their similarity scores and retrieves up to four relevant sections. Chunks with zero similarity are excluded from normal question retrieval.

Answer Generation

The retrieved sections are combined into a context and passed to `Qwen3 4B` through `Ollama`. The model generates a natural-language answer based on the supplied context. It is instructed not to invent unsupported information and to indicate when an answer cannot be found.

User Interface

The Streamlit interface provides document upload, document processing, a chat interface, processing status, conversation history, and expandable source sections. This enables users to interact with the system without directly using command-line tools.

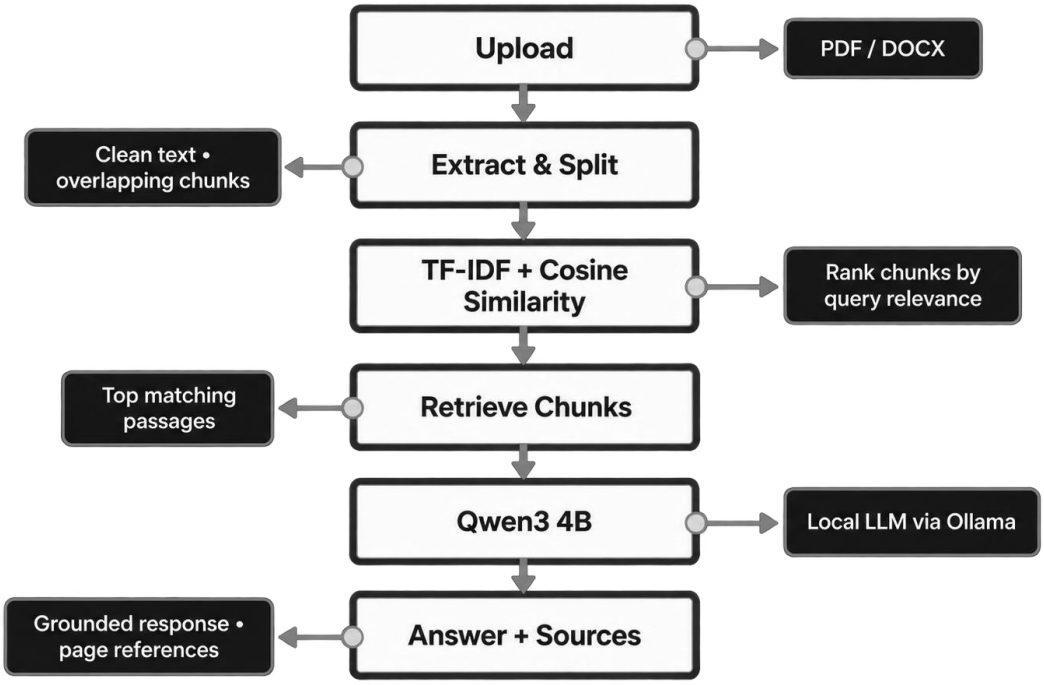
Formulae

$$\boldsymbol{TF}\ (\boldsymbol{i}) = \mathbf{1} + \mathbf{log}(\mathbf{N}(\mathbf{i}))$$

$$\boldsymbol{IDF}\ (\boldsymbol{i}) = \boldsymbol{log}(\boldsymbol{N}/\boldsymbol{N}(\boldsymbol{i}))$$

$$\boldsymbol{COSINE\ SIMILARITY}\ (\boldsymbol{i}) = \frac{\boldsymbol{dot\ product}}{\boldsymbol{magnitude\ of\ document} * \boldsymbol{magnitude\ of\ query}}$$

System Flow



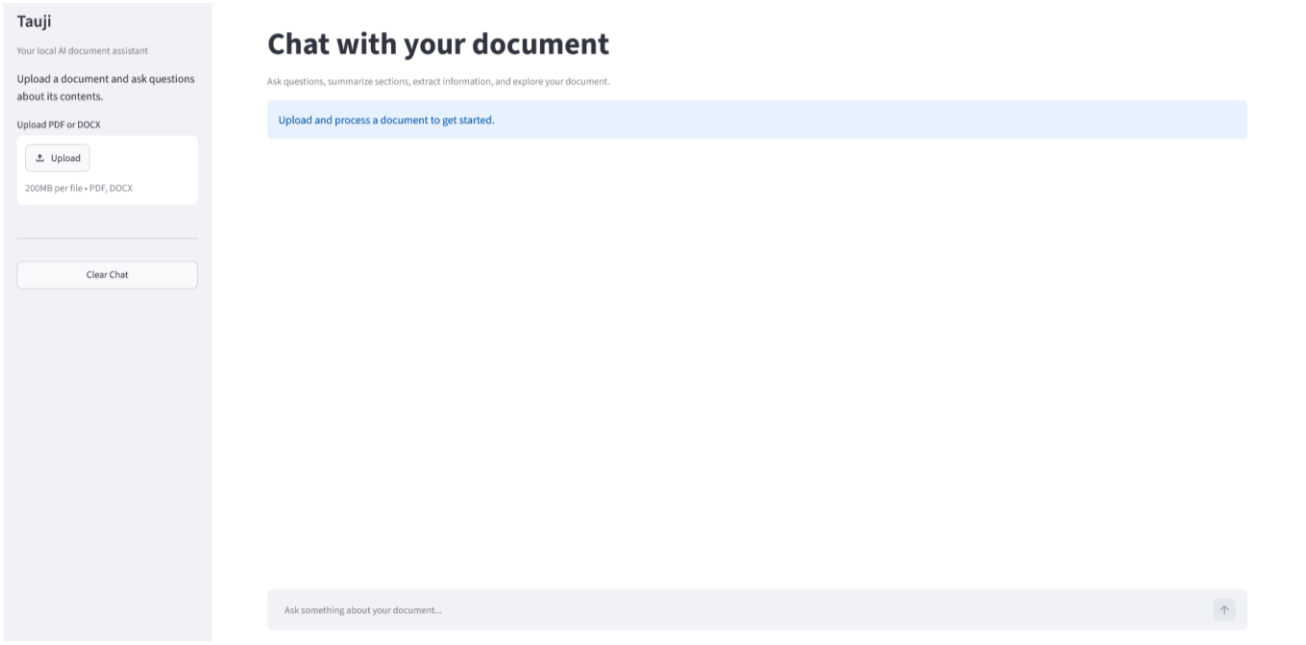
Technology Stack

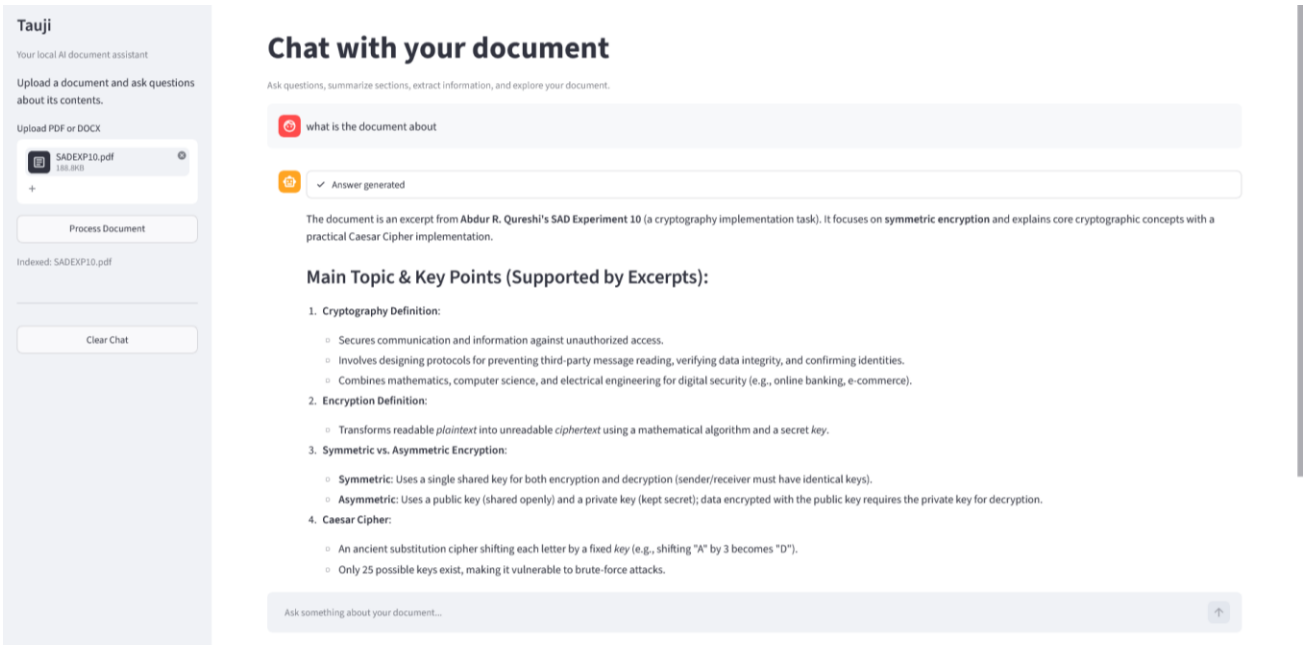
Technology	Purpose
`Python`	Core programming language
`Streamlit`	User interface
`LangChain`	Document loading, chunking, and `LLM` integration
`PyPDFLoader`	`PDF` text extraction
`Docx2txtLoader`	`DOCX` text extraction
`Scikit-learn`	`TF-IDF` vectorization and cosine similarity
`Ollama`	Local language-model execution
`Qwen3 4B`	Answer generation
`Alacritty Bash`	Terminal Emulator
`VS Code`	File Editor

Results

Tauji demonstrates a document question-answering workflow in which users upload a document, ask questions, and receive answers generated from retrieved text sections. The `TF-IDF` retrieval stage avoids the need to generate embeddings through a separate local embedding model, making document indexing lightweight for text-based documents.

Outputs





Github Repository Link: https://github.com/qarq90/tauji.git

Limitations

- TF-IDF primarily relies on word and phrase overlap and may not retrieve semantically related text when different vocabulary is used.
- Answer generation can be slow when Qwen3 4B runs on CPU hardware.
- The current implementation focuses on text and does not directly interpret embedded images or diagrams.
- A summary based on only a few initial chunks may not cover an entire long document.
- The quality of the answer depends on the relevance and completeness of the retrieved context.

Future Scope

- Supporting multiple documents in a single session.
- Adding OCR for scanned PDF files.
- Implementing image and table understanding.
- Improving retrieval with hybrid search or semantic embeddings.
- Adding complete document summarization.
- Implementing persistent storage for document indexes.
- Improving response speed through model and inference optimization.
- Adding conversation-aware retrieval for follow-up questions.

Conclusion

Tauji demonstrates the development of a locally running AI-based Document Question Answering system using retrieval and generative AI. It combines document extraction, text chunking, TF-IDF vectorization, cosine similarity retrieval, and Qwen3 4B answer generation.

The system provides a convenient alternative to manual document searching by allowing users to ask questions in natural language and inspect relevant source sections. Its lightweight retrieval approach and local language-model execution make it suitable for a college mini-project demonstrating the practical application of RAG concepts.

References

- Streamlit Documentation : https://docs.streamlit.io/
- LangChain Documentation : https://python.langchain.com/
- Scikit-learn Documentation : https://scikit-learn.org/
- Ollama Documentation : https://docs.ollama.com/
- Qwen Model Documentation : https://qwen.readthedocs.io/
- Sana Ma’am, *IRS (Information Retrieval System) Numerical (Vector Space Model)*, classroom notes and academic reference material.